OOP Interview Questions

42 unique questions from three sets, with short answers you can actually speak

Duplicates across the three sets are merged into one entry each. Every answer is three to five sentences, which is about twenty to thirty seconds of speaking: definition first, then why it matters, then an example. The italic line below some answers is what to add only if the interviewer probes further. Read them aloud once or twice rather than memorising the wording.

Contents

1. Fundamentals (Q1-Q8)
2. Encapsulation and Abstraction (Q9-Q12)
3. Inheritance and Relationships (Q13-Q20)
4. Polymorphism (Q21-Q25)
5. Abstract Classes and Interfaces (Q26-Q28)
6. Constructors and Destructors (Q29-Q35)
7. Keywords and Special Members (Q36-Q41)
8. Object Copying (Q42-Q42)

1. Fundamentals

Q1. What is Object-Oriented Programming?

Asked in: Set 1, 3

OOP is a way of designing software where I model the system as objects instead of a sequence of functions. Each object keeps its own data and the methods that work on that data together in one unit. It's built on four pillars: encapsulation, abstraction, inheritance and polymorphism. The benefit is that changes stay local to one class, so large systems are easier to maintain and reuse.

Examples: Java, C++, Python, C#.

Q2. Difference between procedural programming and OOP?

Asked in: Set 3

In procedural programming I think in steps: functions call each other and the data sits separately, often global. In OOP I think in entities: data and behaviour are bundled inside objects. So procedural code has no real data protection, while OOP hides data behind the class's own methods. Procedural is fine for small programs; OOP scales much better for large applications.

One line: procedural asks 'what steps do I run', OOP asks 'which objects exist and how do they interact'.

Q3. Advantages and disadvantages of OOP?

Asked in: Set 3

The advantages are reusability through inheritance and composition, easier maintenance because changes stay inside one class, data security through encapsulation, and modularity so teams can work in parallel. The disadvantages are a steeper learning curve, a tendency to over-engineer small problems, and some runtime cost from object creation and virtual dispatch. So OOP pays off on large, long-lived applications and can be overkill for small scripts.

Q4. What are the four pillars of OOP?

Asked in: Set 1, 2, 3

Encapsulation is bundling data with the methods that use it and hiding that data behind controlled access. Abstraction is showing only what an object does and hiding how it does it. Inheritance is deriving a new class from an existing one, which models an 'is-a' relationship and gives reuse. Polymorphism is one interface behaving differently depending on the actual object or the arguments.

Closing line: encapsulation protects state, abstraction simplifies the view, inheritance gives reuse, polymorphism gives flexibility.

Q5. What is a class and what is an object?

Asked in: Set 1, 2, 3

A class is a user-defined type that acts as a blueprint. It declares what data and methods the objects will have, but by itself it holds no values. An object is an actual instance of that class in memory, with its own copy of the data. The class is the blueprint of a house; the object is the actual house built from it, and many houses can come from one blueprint.

Q6. How much memory does a class occupy?

Asked in: Set 3

A class itself takes no memory for data, because it's only a compile-time description. Memory is allocated when I create an object, and each object holds its own non-static data members. Member functions are stored once and shared by all objects, and static members live in one shared location.

If the class has virtual functions, every object also carries a hidden vptr, so it grows by one pointer.

Q7. Is it always necessary to create an object to use a class?

Asked in: Set 3

No. Static members belong to the class itself, so I can use them through the class name without creating any object. Non-static members always need an object, because they operate on that object's state. Utility classes with only static methods are the common example.

Q8. Difference between a struct and a class in C++?

Asked in: Set 3

In C++ they're almost identical: a struct can have member functions, constructors and inheritance just like a class. The only real difference is the default access. Struct members are public by default, class members are private by default. By convention I use struct for plain data holders and class when the type has rules to protect.

2. Encapsulation and Abstraction

Q9. What is encapsulation?

Asked in: Set 1, 3

Encapsulation is wrapping data and the methods that act on it into one unit, the class, and restricting direct access to that data. I do it by making the fields private and exposing public methods to work with them. The benefit is that validation lives in one place, so nobody can put the object into an invalid state. It also means I can change the internal representation later without breaking callers.

Example: a BankAccount keeps balance private, so withdraw can check it never goes negative.

Q10. What is abstraction?

Asked in: Set 1, 3

Abstraction means showing only the essential features and hiding the implementation details. The user knows what the object does, not how. I achieve it with abstract classes and interfaces. The classic example is a car: I need the steering wheel and pedals, not knowledge of how the engine works.

Q11. Difference between encapsulation and abstraction?

Asked in: Set 1, 2, 3

Abstraction hides complexity; encapsulation hides data. Abstraction happens at the design level, where I decide what the outside world should see, and I express it through interfaces. Encapsulation happens at the implementation level, where I make fields private so the state can't be corrupted. They work together: the interface abstracts, and the class behind it encapsulates.

Q12. What are access specifiers and why do they matter?

Asked in: Set 1, 3

Access specifiers control who can access a member of a class, and they're what actually makes encapsulation possible. Public is accessible from anywhere, protected is accessible inside the class and its derived classes, and private only inside the class itself. Java adds a default or package-private level, visible within the same package. My rule is to keep everything as private as possible and expose only the real public API.

3. Inheritance and Relationships

Q13. What is inheritance and what is its purpose?

Asked in: Set 1, 3

Inheritance lets a new class acquire the data and methods of an existing class. The existing one is the base or parent, the new one is the derived or child, and it models an 'is-a' relationship, like a Car is a Vehicle. The purpose is code reuse and, more importantly, enabling runtime polymorphism, since I can hold a Car in a Vehicle pointer.

Caution: use it only when the is-a relationship is genuinely true; otherwise use composition.

Q14. What are the types of inheritance?

Asked in: Set 1, 3

Single, where one class derives from one base. Multilevel, which is a chain: C from B, B from A. Hierarchical, where several classes derive from the same base. Multiple, where one class derives from more than one base, which C++ supports but Java doesn't for classes. And hybrid, which is a combination of these and can cause the diamond problem.

Q15. Are there any limitations of inheritance?

Asked in: Set 3

Yes. It creates tight coupling, so a change in the base class can break every child, which is the fragile base class problem. It also weakens encapsulation, because the subclass sees protected internals. Deep hierarchies get hard to trace, and multiple inheritance brings ambiguity. That's why the usual guideline is to favour composition over inheritance.

Q16. What is the diamond problem and how is it resolved?

Asked in: Set 3

It happens in multiple inheritance when B and C both inherit from A, and D inherits from both B and C. D then ends up with two copies of A's members, so a call to them is ambiguous. In C++ it's solved with virtual inheritance, so only one shared copy of A exists. Java avoids it entirely by allowing only single inheritance of classes.

Q17. Difference between composition and inheritance?

Asked in: Set 1

Inheritance is an 'is-a' relationship, composition is a 'has-a'. A Dog is an Animal, but a Car has an Engine. Inheritance ties the child to the parent at compile time and exposes its internals, so it's tightly coupled, while composition works through the contained object's public interface and stays loose. Composition is also swappable at runtime, so I default to it unless the is-a relationship is genuinely true.

Q18. Difference between association, aggregation and composition?

Asked in: Set 3

They differ in how strong the relationship is. Association is the loosest, where two objects simply know each other and both exist independently, like a Teacher and a Student. Aggregation is a weak has-a where the part survives without the whole, like a Department and its Professors. Composition is a strong has-a with ownership, where the part dies with the whole, like a House and its Rooms.

Q19. What is coupling and why does it matter?

Asked in: Set 2

Coupling is how much one class depends on another. Loose coupling is what we want, because then a change in one class doesn't ripple through the rest, and I can test a class in isolation with mocks. Tight coupling means one small change forces changes everywhere. I reduce it by programming to interfaces, using dependency injection, and preferring composition.

It pairs with cohesion: aim for high cohesion within a class and low coupling between classes.

Q20. What is the Liskov Substitution Principle?

Asked in: Set 1

It's the L in SOLID. It says anywhere I use a base class object, I should be able to substitute a subclass object and the program should still be correct. So a subclass must honour the parent's contract, not strengthen its requirements or break its guarantees. The classic violation is Square inheriting Rectangle: setting the width also changes the height, so code written for Rectangle breaks.

4. Polymorphism

Q21. What is polymorphism and what are its types?

Asked in: Set 1, 3

Polymorphism means one interface taking many forms, so the same call behaves differently depending on the context. There are two types. Compile-time polymorphism, also called early binding, is resolved by the compiler and comes from method overloading and operator overloading. Runtime polymorphism, or late binding, is resolved while the program runs and comes from method overriding through virtual functions.

Q22. Difference between overloading and overriding?

Asked in: Set 1, 2, 3

Overloading is having several methods with the same name in the same class but different parameter lists, and the compiler picks one based on the arguments, so it's compile-time. Overriding is when a derived class gives its own implementation of a base class method with the same signature, and which one runs is decided at runtime from the actual object type. In C++ the base method has to be virtual for overriding to work.

Short version: overloading is same name, different parameters, same class; overriding is same name, same parameters, across base and derived.

Q23. What is dynamic method dispatch?

Asked in: Set 1

It's the mechanism behind runtime polymorphism. When I call an overridden method through a base pointer, the version that runs is chosen at runtime from the actual object type, not the pointer type. C++ implements it with a vtable per class and a hidden vptr in each object, so the call looks up the function address and jumps there. In Java every ordinary instance method works this way by default.

Q24. What are virtual and pure virtual functions?

Asked in: Set 1, 2, 3

A virtual function is a base class method that derived classes can override, and when called through a base pointer the actual object's version runs. In C++ I mark it with the virtual keyword; in Java all instance methods are virtual by default. A pure virtual function has no implementation in the base class and is declared with equals zero, which makes the class abstract and forces derived classes to implement it.

Important: a polymorphic base class must have a virtual destructor, or deleting through a base pointer won't run the derived destructor.

Q25. What is method hiding?

Asked in: Set 1

Method hiding is when a derived class declares a method with the same name as one in the base, but the call is still resolved at compile time from the reference type. In Java this happens with static methods: the child's static method hides the parent's, so a parent reference pointing to a child object calls the parent's version. The contrast with overriding is that overriding is resolved at runtime by the object type.

5. Abstract Classes and Interfaces

Q26. What is an abstract class?

Asked in: Set 1, 2, 3

An abstract class is meant to be inherited from and cannot be instantiated on its own. It can have both abstract methods with no body and normal concrete methods, plus data members and constructors. In C++ a class becomes abstract when it has at least one pure virtual function; in Java I use the abstract keyword. Derived classes must implement all the pure virtual functions, otherwise they stay abstract too.

Q27. What is an interface and what is its purpose?

Asked in: Set 1, 2, 3

An interface is a contract: it declares methods that any implementing class must provide, with no implementation of its own. Its purpose is abstraction, polymorphism so I can write code against the interface and it works with any implementer, and multiple inheritance of type in languages where a class can extend only one class. It also gives loose coupling, which is what makes dependency injection and testing with mocks possible.

C++ has no interface keyword; there it's an abstract class where every function is pure virtual.

Q28. Difference between an abstract class and an interface?

Asked in: Set 2, 3

An abstract class can have both abstract and implemented methods, plus instance variables and constructors. An interface mainly just declares a contract and can't hold instance state. In Java a class can extend only one abstract class but implement many interfaces. So I use an abstract class when related classes share code and state, and an interface when unrelated classes need to follow the same contract.

Example: Bird and Aeroplane are unrelated, but both can implement a Flyable interface.

6. Constructors and Destructors

Q29. What is a constructor?

Asked in: Set 1, 3

A constructor is a special member function called automatically when an object is created, and its job is to initialise the object into a valid state. It has the same name as the class and no return type, which is what separates it from a normal method. If I don't write one, the compiler generates a default constructor, but that disappears as soon as I declare any constructor myself.

Q30. What are the types of constructors?

Asked in: Set 2, 3

There are three main ones. The default constructor takes no arguments and is what runs when I create an object without passing anything. The parameterized constructor takes arguments and initialises members with those values. The copy constructor creates a new object as a copy of an existing one, and takes a const reference to the same class.

C++11 also adds the move constructor, which takes an rvalue reference and steals resources from a temporary instead of copying.

Q31. Can we overload a constructor?

Asked in: Set 3

Yes. Constructor overloading just means having several constructors with different parameter lists, and the compiler picks one at compile time from the arguments. That's how I let the same class be created in different ways, like a Date with no arguments giving today, or with three integers giving a specific date.

Q32. What is constructor chaining?

Asked in: Set 1

Constructor chaining is one constructor calling another, so common initialisation is written once. Within the same class it's this with arguments in Java, or a delegating constructor in C++11. Across a hierarchy, the derived constructor calls the base constructor, using super in Java or the base class name in the C++ initializer list. The base constructor always runs first, before the derived constructor body.

Q33. Difference between a public and a private constructor?

Asked in: Set 2

A public constructor can be called from anywhere, so any code can create objects. A private constructor can only be called from inside the class, which means the class controls how objects are created. The classic use is the singleton pattern, where the constructor is private and a static method hands out the one instance. It's also used for static factory methods and utility classes that should never be instantiated.

Q34. What is a destructor?

Asked in: Set 1, 2, 3

A destructor is called automatically when an object is destroyed, either when it goes out of scope or when I delete it, and its job is to release resources like memory, file handles or connections. In C++ it has the class name with a tilde in front, takes no parameters and returns nothing. Java has no destructors because the garbage collector handles memory.

Two follow-ups: this is the basis of RAII in C++, and a polymorphic base class needs a virtual destructor.

Q35. Can we overload a destructor?

Asked in: Set 3

No. Overloading needs different parameter lists, and a destructor takes no parameters and returns nothing, so there's nothing to vary. A class can have only one destructor.

7. Keywords and Special Members

Q36. What is the this keyword?

Asked in: Set 1, 2

this is a pointer, or a reference in Java, to the object the current method is running on. The most common use is disambiguation, when a parameter has the same name as a member, so I write this->name for the member. It's also used to return the current object for method chaining, or to pass the object itself to something else. Static methods have no this, which is why they can't touch non-static members.

Q37. What is the super keyword?

Asked in: Set 1

super refers to the immediate parent class. I use it to call the parent's constructor, to call the parent's version of a method I've overridden so I can extend rather than replace it, and to access a parent field the child has hidden. C++ has no super keyword; there I qualify the call with the base class name, like Base::show.

Q38. What is the static keyword, and how do static and non-static members differ?

Asked in: Set 1, 2, 3

A static member belongs to the class itself, not to any object. There's one shared copy for all objects and it's accessed through the class name, so it's useful for things like a counter of how many objects exist. A static method can be called without an object, and because it has no this pointer it can only use static members directly. A non-static method is called on an object and can use both static and non-static members.

Q39. What does the final keyword do?

Asked in: Set 1

In Java, final means something can't be changed further. A final variable can be assigned only once, a final method can't be overridden, and a final class can't be extended, which is why String is final. In C++ the equivalents are const for values, and the final specifier on a virtual function or class to stop overriding or inheritance.

Q40. What is the instanceof operator used for?

Asked in: Set 1

instanceof checks at runtime whether an object is an instance of a class, a subclass, or an implementer of an interface, and returns a boolean. The usual use is before a downcast, so I don't get a ClassCastException. The C++ equivalent is dynamic_cast, which returns null when the cast fails.

Caution: lots of instanceof checks usually means the design should use polymorphism instead.

Q41. What are friend functions and friend classes?

Asked in: Set 3

A friend function is a non-member function given access to a class's private and protected members, and a friend class gets that access for all its methods. Both are declared inside the class with the friend keyword. The typical use is operator overloading, especially the stream insertion operator. Friendship isn't mutual, isn't inherited and isn't transitive, and since it breaks encapsulation it should be used sparingly.

8. Object Copying

Q42. Difference between a shallow copy and a deep copy?

Asked in: Set 1, 2, 3

The difference is what happens to the data the object points to. A shallow copy copies members as they are, so a pointer member is copied but not the data behind it, and both objects end up sharing that data. A deep copy allocates new memory and copies everything recursively, so the two objects are completely independent. In C++ the compiler-generated copy constructor is shallow, so if my class owns a raw pointer I have to write a deep copy myself, otherwise both destructors free the same memory.

That's the Rule of Three: if I need a destructor, I also need a copy constructor and copy assignment. Better still, use smart pointers or std::vector so the defaults are already correct.